

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Kishore Gadhi	Reg. No.:	192365051
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION C – APPLICATION BASED QUESTIONS

Master Theorem Application for Equal Growth Case

Consider the recurrence relation $T(n) = 2T(n/2) + n$. Analyze the problem using the Master Theorem and determine its time complexity. Clearly identify the values of a , b , and $f(n)$, and compare $f(n)$ with $n^{\log_b a}$. Justify the selected case step-by-step and explain why it belongs to the equal-growth category. Use recursion tree support if needed and present the final result with proper interpretation.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Master Theorem Application for Equal Growth Case

Consider the recurrence relation $T(n) = 2T(n/2) + n$. Analyze the problem using the Master Theorem and determine its time complexity. Clearly identify the values of a , b , and $f(n)$, and compare $f(n)$ with $n^{\log_b a}$. Justify the selected case step-by-step and explain why it belongs to the equal-growth category. Use recursion tree support if needed and present the final result with proper interpretation.

Geo Tag Photo:

s



Power point Presentation:

ALGORITHM OVERVIEW

A cookbook method for solving divide-and-conquer recurrences of the form $T(n) = aT(n/b) + f(n)$ where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive.

I	Leaf-Heavy
---	------------

$$f(n) = O(n^{(\log_b a - \epsilon)})$$

$f(n)$ grows slower than $n^{(\log_b a)}$.
Leaves dominate. Solution:

$$T(n) = \Theta(n^{\log_b a})$$

Equal Growth

SELECTED

$$f(n) = \Theta(n^{\log_b a})$$

$f(n)$ and $n^{(\log_b a)}$ grow at the same rate. Every level contributes equally. Solution:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

III Root-Heavy

$$f(n) = \Omega(n^{(\log_b a + \epsilon)})$$

$f(n)$ grows faster than $n^{(\log_b a)}$.
Root dominates. Solution:

$$T(n) = \Theta(f(n))$$

APPR: _____

SCALE: NTS

ALGORITHM OVERVIEW

A cookbook method for solving divide-and-conquer recurrences of the form $T(n) = aT(n/b) + f(n)$ where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive.

I Leaf-Heavy

$$f(n) = O(n^{(\log_b a - \epsilon)})$$

$f(n)$ grows slower than $n^{(\log_b a)}$.
Leaves dominate. Solution:

$$T(n) = \Theta(n^{\log_b a})$$

Equal Growth

SELECTED

$$f(n) = \Theta(n^{\log_b a})$$

$f(n)$ and $n^{\log_b a}$ grow at the same rate. Every level contributes equally. Solution:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

III Root-Heavy

$$f(n) = \Omega(n^{(\log_b a + \epsilon)})$$

$f(n)$ grows faster than $n^{(\log_b a)}$.
Root dominates. Solution:

$$T(n) = \Theta(f(n))$$

APPR: _____ +

SCALE: NTS

ALGORITHM OVERVIEW

FIG. 03 - MASTER THEOREM STRUCTURE

VAR: RECURRENCE
STATUS: ACTIVE

WHAT IS THE MASTER THEOREM?

A cookbook method for solving divide-and-conquer recurrences of the form $T(n) = aT(n/b) + f(n)$ where $a \geq 1$, $b > 1$, and $f(n)$ is asymptotically positive.

I Leaf-Heavy

$$f(n) = O(n^{\log_b a - \epsilon})$$

$f(n)$ grows slower than $n^{\log_b a}$.
Leaves dominate. Solution:

$$T(n) = \Theta(n^{\log_b a})$$

II Equal Growth

SELECTED

$$f(n) = \Theta(n^{\log_b a})$$

$f(n)$ and $n^{\log_b a}$ grow at the
same rate. Every level contributes
equally. Solution:

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

III Root-Heavy

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

$f(n)$ grows faster than $n^{\log_b a}$.
Root dominates. Solution:

$$T(n) = \Theta(f(n))$$

DRAWN BY: ALGORITHMS DIVISION

DATE: 2026

SHEET: 03/10

APPR: _____

SCALE: NTS

ALGORITHM / PSEUDOCODE

FIG. 04 - FORMAL THEOREM + CASE II LOGIC

TYPE: THEOREM
STATUS: ACTIVE

MASTER THEOREM - FORMAL DEFINITION

PSEUDO.01

Given: $T(n) = aT(n/b) + f(n)$
Let: $p = \log_b(a)$ // critical exponent

// — Case I —
IF $f(n) = O(n^{p - \epsilon})$ // $\epsilon > 0$
THEN $T(n) = \Theta(n^p)$

// — Case II — SELECTED —
IF $f(n) = \Theta(n^p)$
THEN $T(n) = \Theta(n^p \cdot \log n)$

// — Case III —
IF $f(n) = \Omega(n^{p + \epsilon})$ // + regularity
THEN $T(n) = \Theta(f(n))$

CASE II - STEP-BY-STEP CHECK

STEP 01 - COMPUTE p
 $p = \log_b(a) = \log_2(2) = 1$

STEP 02 - FORM n^p
 $n^p = n^1 = n$

STEP 03 - COMPARE $f(n)$
 $f(n) = n = \Theta(n^1) = \Theta(n^p) \checkmark$

CONCLUSION - APPLY CASE II
 $T(n) = \Theta(n \log n)$

DRAWN BY: ALGORITHMS DIVISION

DATE: 2026

SHEET: 04/10

APPR: _____

SCALE: NTS

WORKING EXAMPLE

FIG. 05 - DRY RUN: $T(n) = 2T(n/2) + n$, $n=8$ INPUT: $n = 8$
STATUS: TRACED

STEP-BY-STEP TRACE

- Identify parameters**
 $a = 2$, $b = 2$, $f(n) = n$
- Compute critical exponent**
 $p = \log_b(a) = \log_2(2) = 1$
- Compare $f(n)$ to n^p**
 $f(n) = n$ vs $n^1 = n \rightarrow$ equal rate
- Conclude Case II applies**
 $f(n) = \Theta(n^p)$, so multiply by $\log n$

FINAL RESULT

$$T(n) = \Theta(n \log n)$$

DIAGRAM A.2 - EXPANSION TREE ($n=8$)

DRAWN BY: ALGORITHMS DIVISION

DATE: 2026

SHEET: 05/10

SCALE: NTS

COMPLEXITY ANALYSIS

FIG. 06 - RECURSION TREE COST DERIVATION

UNITS: ASYMPTOTIC
STATUS: VERIFIED

RECURSION TREE - COST ANALYSIS

CHART 2.0

LEVEL	SUBPROBLEMS	SIZE	LEVEL COST
0	$2^0 = 1$	n	n
1	$2^1 = 2$	$n/2$	$2 \times n/2 = n$
2	$2^2 = 4$	$n/4$	$4 \times n/4 = n$
k	2^k	$n/2^k$	$2^k \times n/2^k = n$
$\log n$	n (leaves)	1	$n \times \Theta(1)$

KEY METRICS

TREE DEPTH

 $\log n$ levels until $T(1)$

COST PER LEVEL

 n

uniform at every level

TOTAL COST

 $n \log n$ $\log n$ levels $\times n$ per level

ASYMPTOTIC BOUND

 $T(n) = \Theta(n \log n)$

APPR: _____

SCALE: NTS

COMPARATIVE ANALYSIS

FIG. 07 - ALL THREE CASES SIDE-BY-SIDE

DATA SET: MASTER-THM
STATUS: COMPLETE

CASE	CONDITION	GROWTH TYPE	EXAMPLE	DOMINATES	RESULT
I	$f(n) = O(n^{(p-\epsilon)})$	Sublinear combine	Binary search	Leaf nodes	$\Theta(n^p)$
II	$f(n) = \Theta(n^p)$	Equal growth rate	Merge sort	All levels equal	$\Theta(n^p \cdot \log n)$
III	$f(n) = \Omega(n^{(p+\epsilon)})$	Superlinear combine	Matrix multiply	Root node	$\Theta(f(n))$

Case II key insight: every level of the recursion tree contributes exactly n work — no level dominates. The $\log n$ factor counts the number of levels.

OUR CASE: $T(n) = \Theta(n^1 \cdot \log n) = \Theta(n \log n)$

DRAWN BY: ALGORITHMS DIVISION

DATE: 2026

SHEET: 07/10

APPR: _____
SCALE: NTS

APPLICATIONS

FIG. 08 - REAL-WORLD CASE II ALGORITHMS

DOMAIN: CS ALGORITHMS
STATUS: VERIFIED

APP_01

Merge Sort

$$T(n) = 2T(n/2) + n$$

Splits array into halves recursively; linear-time merge step. The canonical Case II algorithm. Stable, predictable, widely used in standard libraries.

Result: $\Theta(n \log n)$

APP_02

Heap Sort

$$T(n) = 2T(n/2) + O(n)$$

Build-heap phase processes nodes in a divide-and-conquer pattern. Each heapify call is $O(\log n)$; building a full heap yields $O(n \log n)$ total.

Result: $\Theta(n \log n)$

APP_03

Quick Sort (avg)

$$T(n) \approx 2T(n/2) + n$$

On average with a balanced pivot, partitioning evenly halves the problem. The linear partition step at each level leads to Case II behavior.

Avg: $\Theta(n \log n)$

APP_04

Fast Fourier Transform

$$T(n) = 2T(n/2) + n$$

Cooley-Tukey FFT recursively splits the DFT in half; the butterfly combine stage costs $O(n)$ per level. A cornerstone of signal processing.

Result: $\Theta(n \log n)$

DRAWN BY: ALGORITHMS DIVISION

DATE: 2026

APPR: _____
SCALE: NTS SHEET: 08/10

RESULTS & DISCUSSION

FIG. 09 - VERIFIED OUTCOME + INTERPRETATION

CONF: HIGH
STATUS: FINAL

VERIFIED RESULT

$$T(n) = \Theta(n \log n)$$

CASE II - EQUAL GROWTH CONFIRMED

VERIFICATION CHECKLIST

- + $a = 2, b = 2$ confirmed - valid Master Theorem form
- + $p = \log_2(2) = 1$, so $n^p = n^1 = n$
- + $f(n) = n = \Theta(n) = \Theta(n^p)$ - Case II condition met

DISCUSSION POINTS

Why log n appears

Each of the $\log n$ levels of the recursion tree contributes n work. None dominates - hence the accumulated factor of $\log n$ multiplied by n .

Optimality of Merge Sort

The information-theoretic lower bound for comparison-based sorting is $\Omega(n \log n)$. Merge Sort's $\Theta(n \log n)$ is therefore optimal in the worst case.

Practical significance

$n \log n$ grows only slightly faster than n . For $n = 10^6$, $\log n \approx 20$ - a modest constant overhead, explaining why such algorithms scale well in practice.

Big-O vs Theta distinction

The Master Theorem yields Theta (tight bound), confirming both the upper and lower bound for the recurrence, which is a stronger result.

APPR: not just O -
for the recurrence,
SCALE: NTS

CONCLUSION & REFERENCES

FIG. 10 - SUMMARY + CITED WORKS

STATUS: COMPLETE
REV: FINAL

KEY TAKEAWAYS

- Master Theorem resolves D&C recurrences**
 $T(n) = aT(n/b) + f(n)$ is solved by comparing $f(n)$ to the critical threshold $n^{\log_b a}$ without expanding the recursion manually.
- Case II is the equal-growth scenario**
When $f(n) = \Theta(n^p)$, all $\log n$ levels of the recursion tree contribute equally, yielding a tight bound of $\Theta(n^p \cdot \log n)$.
- $T(n) = 2T(n/2) + n = \Theta(n \log n)$**
Confirmed by parameter extraction ($a=2, b=2, p=1$) and comparison of $f(n)=n$ with $n^1=n$ - an exact match satisfying Case II.
- This is an optimal bound for sorting**
 $\Theta(n \log n)$ matches the information-theoretic lower bound for comparison-based sorting, making Merge Sort asymptotically optimal.

ESTABLISHED RESULT

$$T(n) = 2T(n/2) + n$$

$$\Theta(n \log n)$$

MASTER THEOREM - CASE II

REFERENCES

- [1] Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C. - Introduction to Algorithms, 4th ed. MIT Press, 2022.
- [2] Akra, M., Bazzi, L. - On the Solution of Linear Recurrence Equations. Computational Optimization, 1998.
- [3] Skiena, S.S. - The Algorithm Design Manual, 3rd ed. Springer, 2020.
- [4] Kleinberg, J., Tardos, E. - Algorithm Design. Pearson, 2005.

APPR: _____

SCALE: NTS